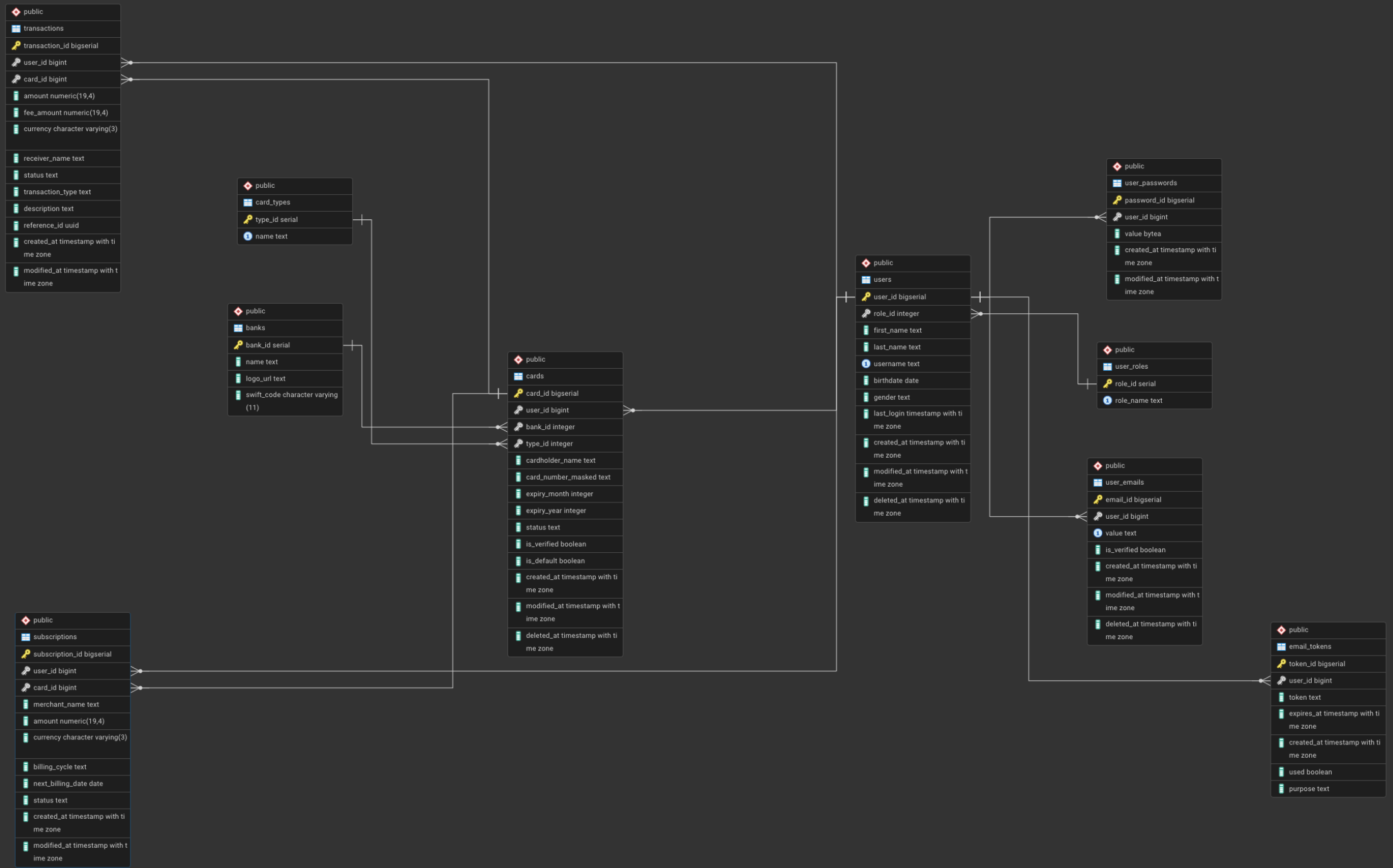
public
transactions
transaction_id bigserial
user_id bigint
card_id bigint
amount numeric(19,4)
fee_amount numeric(19,4)
currency character varying(3)
receiver_name text
status text
transaction_type text
description text
reference_id uuid
created_at timestamp with time zone
modified_at timestamp with time zone
public
card_types
type_id serial
name text
public
banks
bank_id serial
name text
logo_url text
swift_code character varying (11)
public
cards
card_id bigserial
user_id bigint
bank_id integer
type_id integer
cardholder_name text
card_number_masked text
expiry_month integer
expiry_year integer
status text
is_verified boolean
is_default boolean
created_at timestamp with time zone
modified_at timestamp with time zone
deleted_at timestamp with time zone
public
users
user_id bigserial
role_id integer
first_name text
last_name text
username text
birthdate date
gender text
last_login timestamp with time zone
created_at timestamp with time zone
modified_at timestamp with time zone
deleted_at timestamp with time zone
public
user_passwords
password_id bigserial
user_id bigint
value bytea
created_at timestamp with time zone
modified_at timestamp with time zone
public
user_roles
role_id serial
role_name text
public
user_emails
email_id bigserial
user_id bigint
value text
is_verified boolean
created_at timestamp with time zone
modified_at timestamp with time zone
deleted_at timestamp with time zone
public
email_tokens
token_id bigserial
user_id bigint
token text
expires_at timestamp with time zone
created_at timestamp with time zone
used boolean
purpose text
public
subscriptions
subscription_id bigserial
user_id bigint
card_id bigint
merchant_name text
amount numeric(19,4)
currency character varying(3)
billing_cycle text
next_billing_date date
status text
created_at timestamp with time zone
modified_at timestamp with time zone

Database Architecture Overview

The schema is designed using a high degree of normalization to ensure data integrity and scalability for a fintech MVP. By decoupling core entities into specialized tables, the system avoids data redundancy and maintains a clear separation of concerns.

Core User Management

The architecture follows a hub-and-spoke model with the `users` table at the center.

- **Authentication and Identity:** Passwords and emails are stored in separate tables (`user_passwords`, `user_emails`). This allows for a multi-email support system where one address is flagged as the primary contact. Storing passwords separately enables a secure password history log without cluttering the main profile table.
- **Access Control:** The `user_roles` table defines permissions, allowing for role-based access control (RBAC) that can be expanded without modifying the user schema.
- **Verification Logic:** The `email_tokens` table serves as a centralized engine for One-Time Passwords (OTP). By utilizing a purpose column, the same logic handles registration, password resets, and sensitive transaction approvals.

Card and Banking Integration

The card module prioritizes security and UI flexibility through normalization.

- **Lookup Tables:** The `banks` and `card_types` tables act as reference catalogs. This ensures that the `cards` table only stores IDs, while the frontend can dynamically fetch bank logos and brand assets based on those relationships.
- **Security Standards:** Following security best practices, the system never stores full card numbers. The masked number field only retains the last four digits, ensuring that sensitive financial data is not at risk even if the database is compromised.

Financial Operations

This module manages the flow of funds through two distinct lenses: history and scheduling.

- **Transaction Integrity:** The `transactions` table uses the `NUMERIC(19, 4)` data type to prevent the rounding errors common with floating-point numbers. It logs all finalized or attempted movements of money.
- **Subscription Logic:** While transactions represent historical data, the `subscriptions` table stores rules for the future. It manages recurring billing cycles and allows users to cancel or pause services without affecting historical transaction records.
- **Relational Persistence:** Transactions use a set-null approach for card references. This ensures that if a user deletes a card, their financial history remains intact for reporting and dashboard statistics.

Constraints and Performance

Beyond basic table structures, the schema implements business logic at the database level:

- **Partial Unique Indexes:** These enforce rules directly in the engine, such as ensuring a user can only have one default card and one primary email at any given time.
- **Soft Deletes:** The inclusion of deletion timestamps across major tables allows for data recovery and maintains the integrity of foreign key relations in historical logs.
- **Audit Trails:** Automated timestamps provide a clear record of every creation and modification within the system.

Disclaimer

This ERD serves as a foundational baseline for the MVP. Depending on evolving production requirements, performance tuning, or specific business logic, this schema can be updated, expanded, optimized, or reduced. It is intended to be a flexible structure that adapts to the technical needs of the platform.